

FOE-Dreamer

The lexicon, tiered

The census said what I already write. This says what each term means, what it is not, and how to decide whether it applies to a case in front of me. Thirty-one Active entries carry all four fields, because those are the ones I have to define on the spot in a talk or a response. Working entries carry a definition and the discrimination. Peripheral entries carry a definition and a pointer.

Eight terms are used in two senses across my own files. Those are not tidied away into an errata. Each one sits inside the entry it belongs to, in a marked box, because a term whose two uses disagree is the entry most worth having.

Contents

1	Active	2
1.1	The factoring	2
1.2	The opponent model	5
1.3	The environment	7
1.4	The evaluation	11
1.5	Where the model is wrong	13
2	Working	15
3	Peripheral	17
4	The two checks	19
4.1	Discrimination, run in both directions	19
4.2	Where two locators disagree	20
5	What crosses into the shared lexicon	21
6	Sources	21

1 Active

1.1 The factoring

factored world model

“FOE-Dreamer separates three sources of variation in the latent state: a controllable component driven by defender actions, an exogenous component driven by benign user activity, and an opponent embedding inferred from defender-observable signals alone.”

main.tex, abstract

Definition: A Dreamer-family latent world model whose belief is split into three components, each capturing a distinct source of variation in the transition kernel and the reward.

Distinguishes it from: A monolithic-state world model, which carries one recurrent latent. The split is not a penalty added to a single latent. It is two posterior heads and two transition heads, only one of which sees the action.

Operational test: Take the previous action out of one branch. If nothing in the architecture changes, the model was never factored.

In my project: Section 5.2. The decoder reads z_t and u_t ; the reward predictor reads z_t , u_t and τ ; no shared transition head joins the two factors.

Does the opponent embedding reach the observation decoder

The paper: the decoder is “ $\hat{o}_t = g_\eta(z_t, u_t)$ ”, with no τ . Only the reward predictor takes it: “ $\hat{r}_t = h_\zeta(z_t, u_t, \tau)$ ”, main.tex Factored World Model.

The website: “ z_t^{opp} , z_t, h_t and u_t are joined and read by Dec_o and Dec_r , which reconstruct the observation and the reward”, website FOE-Dreamer page.

Reviewer B7 asks the question in those words, whether the representation “affects predicted observations or only predicted rewards”. My two files give opposite answers, so the figure and the equations have to be brought to one.

controllable latent, z_t

“The controllable latent z_t carries host states whose evolution is action-conditioned: isolation status, deception deployment, observable reboots, and the alert dynamics that follow defender intervention.”

main.tex, Approach, Preliminaries

Definition: As above, and it is the only latent the policy gradient updates.

Distinguishes it from: The exogenous latent u_t . Both are stochastic latents over the same observation. Only z_t ’s posterior and its transition condition on the previous defender action.

Operational test: Does the previous defender action enter the head that produces it? Yes puts the variable in z_t .

In my project: Section 6.4. A multinomial classifier recovers the defender’s action from z_t at 0.71 accuracy against a 0.17 random baseline, and recovers user activity from it at R^2 0.14 to 0.18.

exogenous latent, u_t

“The exogenous latent u_t carries user activity, service load, periodic maintenance, and other observation-affecting variation that neither the defender nor the attacker controls.”

main.tex, Approach, Preliminaries

Definition: As above. Its posterior and its transition are action-free, and policy gradients are detached from it so the policy cannot react to variation it cannot influence.

Distinguishes it from: Two neighbours, and both matter. From z_t , by the action gate. From the opponent embedding τ , which is also uncontrollable and also reward-relevant: the action and the reward alone cannot tell user variation from attacker variation, and a third signal is what separates them.

Operational test: Could a defender action change it? If yes, it is not exogenous. The honest answer here is that exogeneity is imperfect: isolating a host drops user sessions and a reboot terminates them, so the defender’s containment actions do reach the user process.

In my project: Section 6.4, R^2 0.72 under U-rand and 0.79 under U-burst. Failure case (iii) is the one place it visibly breaks: when GHOSTS shifts to burst regime mid-episode, u_t briefly tracks the controllable latent.

opponent embedding, τ

“The opponent embedding τ encodes attacker type and recent attacker behaviour, inferred only from defender-observable signals.”

main.tex, Approach, Preliminaries

Definition: As above. A recurrent encoder over a sliding window of K recent observations, and during training the defender actions over the same window, produces it.

Distinguishes it from: The exogenous latent u_t . Both are uncontrollable. τ is the one carrying a third auxiliary signal, the privileged attacker labels, and that supervision is what makes the other two factors separable at all.

Operational test: Is the quantity recoverable from the recent observation window with no ground-truth feed at inference time? If it needs the feed, it is a label, not an embedding.

In my project: Section 5.3, $K = 10$ and dimension 64. Figure 6 shows the two attacker profiles occupying separated regions of the embedding space.

The opponent embedding has two names and two inputs

The paper calls it τ_t and computes it from observations and actions: “ $\tau_t = f_\xi(o_{t-K:t}, a_{t-K:t-1})$ ”, main.tex Opponent Model.

The website calls it z_t^{opp} and computes it from latents: “A window of recent latent-action pairs runs through a recurrent encoder into z_t^{opp} ”, website FOE-Dreamer page.

Observations and latents are not the same input, and the difference decides whether the embedding can be recomputed inside an imagined rollout. One of the two is wrong.

the factor boundary

“The factor boundary $z_t \perp u_t \mid h_t$ is enforced by architecture, separate posterior heads and separate

transition heads, rather than by post-hoc regularisation.”

main.tex, Approach, Preliminaries

Definition: The conditional independence the model is built to hold, and the statement of what the factoring claims.

Distinguishes it from: Two neighbours. The architectural commitment, which is the single asymmetry that produces the boundary: the boundary is the claim, the commitment is the mechanism. And a testable check on the factoring, which measures whether the boundary held rather than asserting that it does.

Operational test: Is there a shared transition head? If the only path from one factor to the other runs through the decoder and the reward predictor, and both are read-only with respect to the transitions, the boundary holds by construction rather than by tuning.

In my project: Section 5.1 states it; Section 6.4 measures it.

the architectural commitment

“The asymmetry is the architectural commitment of the factoring: the action gate enters z_t alone.”

main.tex, Factored World Model

Definition: The one asymmetry the whole design turns on. The controllable posterior conditions on the previous defender action and the exogenous posterior does not.

Distinguishes it from: The factor boundary, which is the independence statement this sentence delivers.

Operational test: Name the one line of the architecture that would have to change for the factoring to disappear. If more than one line is needed, the commitment has not been isolated.

In my project: Section 5.2. This is the shortest statement of the design I have, and the sentence to reuse when a reviewer asks what the factoring actually is.

spurious credit assignment

“a factored world model that disentangles defender-driven, user-driven, and attacker-driven sources of variation in the latent state, reducing the spurious credit assignment that monolithic-state RL incurs in operational networks”

main.tex, Introduction, first contribution

“If an alert spike coincides with both a user logging in and a defender Isolate action, an unfactored latent has no architectural reason to attribute the spike to one or the other.”

main.tex, Factored World Model

Definition: Credit the policy receives for state changes it did not cause, which a single recurrent latent has no architectural reason to withhold.

Distinguishes it from: Reward noise. Noise averages out across seeds. This does not, because the misattribution runs in the direction of whatever co-occurs with the action.

Operational test: Take a coincidence in the observation stream and ask which component of the model would carry each cause. If the answer is the same component for all of them, the credit

is spurious by construction.

In my project: First contribution, Introduction. The stated cost is that the variance of the policy gradient inflates; the ablation reports it as a 14-point loss against Avoidant and a wider seed band.

identifiability of the adversary latent

“One assumption may not hold: the adversary latent may not be identifiable from defender-side observation at all. Against an adversary living off the land it may be recoverable only up to the defender’s own intervention power, and under a coarse action space not at all.”

website, FOE-Dreamer page

Definition: Whether the intended decomposition is recoverable from the data at all, rather than whether one trained run happened to land near it.

Distinguishes it from: Two neighbours. The fallback, which is what is kept when this assumption fails. And a testable check on the factoring: that measurement reports where one run landed, while identifiability asks whether any run could have landed elsewhere and fit the data equally well.

Operational test: Name the auxiliary variable that separates the two uncontrollable processes. The action separates controllable from uncontrollable, and the reward separates reward-relevant from irrelevant; user activity and attacker activity are uncontrollable and reward-relevant both, so without a third signal they are one block, and u_t is identifiable as uncontrollable reward-relevant variation rather than as user activity.

In my project: Website, FOE-Dreamer page. Reviewer A3 asks for exactly this, that the factoring “only makes sense under certain assumptions, which could be mentioned”. The strongest sentence I have written on it is on the website and not in the paper.

1.2 The opponent model

signatures

“The defender cannot directly observe attacker actions or intent, but attacker behaviour leaves signatures in the defender’s telemetry.”

main.tex, Opponent Model

Definition: Traces attacker behaviour leaves in defender telemetry, which are not visible in any single observation but cluster by attacker class over a short horizon.

Distinguishes it from: Probe-and-withdraw, which is the behaviour; signatures are what it leaves behind. And from an intrusion-detection signature, which matches a pattern inside one event. This one is a statistical shape over a window.

Operational test: Is it visible in a single observation? If yes, it is an alert, not a signature.

In my project: Section 5.3. An Avoidant attacker withdraws when a honeypot triggers; an Enticed attacker presses on against the same signal.

privileged-information training

“During training the testbed exposes ground-truth attacker class and the attacker’s action sequence as privileged signals.”

main.tex, Opponent Model

“At test time only the encoder f_ξ is retained; the privileged labels are not available, and τ_t is computed from defender-observable signals alone.”

main.tex, Opponent Model

Definition: Supervision available from the testbed during training and absent at inference, used to shape a representation that must survive without it.

Distinguishes it from: Matched compute, which is what every method received during training. Privileged information is what only the opponent encoder received, and the two are read as contradicting each other.

Operational test: Is the signal present at inference? If not, name what runs in its place. Here the encoder runs on the same host-level telemetry vector the rest of the system already consumes, with no extra feed.

In my project: Section 5.3.

B5 *“The treatment of privileged information is internally inconsistent”*

Review 724B, comment B5

The reviewers’ own words for what they wanted checked were baseline fairness and equivalent supervision. Sections 5.3 and 6.1 have to say the same thing.

The website gives the opponent model two decoders the paper does not have

The website: “Two decoders read it: Dec_{act} predicts the opponent’s next actions, Dec_{obs} predicts what the opponent will observe.”

The paper has “a small classifier head on τ_t ” supervised against attacker class, main.tex Opponent Model, and no such decoders.

Reviewer B7 asks precisely this, “which opponent-prediction decoders and losses are actually used”. What the encoder is trained to predict decides what the embedding can carry, so this is not a figure caption problem.

probe-and-withdraw

“The expected effect is earlier containment against probe-and-withdraw adversaries.”

main.tex, Opponent Model

“Avoidant’s probe-and-withdraw behavior creates extended ambiguity windows that the opponent model resolves; without it, the policy commits late or wrong.”

main.tex, Ablations

Definition: The Avoidant decision rule seen from the defender’s side: any SUSPICIOUS outcome from any probe triggers immediate withdrawal from the affected host, so the revealing action never arrives.

Distinguishes it from: Signatures. The behaviour creates the ambiguity window; the signatures are what lets the window be closed early.

Operational test: Does the adversary retreat on an ambiguous cue rather than commit to it? If so, waiting for the revealing action is waiting for something that will not come.

In my project: Section 5.3, and the ablation: removing the opponent head costs 16 points against Avoidant and 10 against Enticed.

fixed-class profiles

“We model two fixed-class profiles that bracket the spectrum of attacker sophistication observed in field studies.”

main.tex, Threat Model

Definition: Scripted attackers whose decision rule does not change in response to the deployed defender. Two of them, chosen to bracket a spectrum rather than to sample it.

Distinguishes it from: Adaptive learning adversaries, which retrain against the deployed defender and are out of scope. The two profiles share the three probes and differ only in the decision rule applied to the outcomes.

Operational test: Does the attacker’s rule change in response to what the defender did last episode? If not, fixed-class.

In my project: Section 4.1.

C8 *“Avoidant and Enticed mainly differ in how they respond to deception signals, which directly matches the defender’s deception actions.”*

Review 724C, comment C8

My word is bracket, and the profiles bracket sophistication. C8 says they vary on the one axis the defender acts on. Both are true of the same design, and the answer is the shape of the design rather than a defence of it.

1.3 The environment

operational network

“The term “operational network” in this paper means an emulated multi-host enterprise environment that runs real software on a real cloud substrate, with sub-10 s polling and behaviorally correlated user activity. It does not mean a production network with real attackers and real users.”

main.tex, Testbed, final paragraph

Definition: Stipulated, as above. The term is narrowed on purpose and the narrowing is part of the definition.

Distinguishes it from: The testbed, which is the artifact this term picks out; and from a production network, which the sentence rules out by name.

Operational test: Does an action taken by the policy change the state of a running virtual machine, and is the user population scripted? Both yes puts you in the stipulated sense.

In my project: Section 4.3, final paragraph. Reviewer A2 asked for the definition to be moved forward, which is a placement change and not a disagreement about the sense.

the boundary between what is real and what is emulated

“Real: the OpenStack substrate, the deployed services (WordPress, SSH, NTP, PostgreSQL, cron), the CVE-bound exploit code paths, the wall-clock 5-second polling, and the operational consequences of defender actions on running VMs (an isolated host actually drops user traffic). Emulated: the user population (GHOSTS personas, scripted), the attacker population (two fixed-class profiles, scripted), and the CVE catalog.”

main.tex, Testbed, final paragraph

Definition: The two lists, given in full, with nothing left to inference. Six items real, three emulated.

Distinguishes it from: A fidelity claim. This is an inventory, so it can be checked item by item rather than argued.

Operational test: For any component, would replacing the emulation with the real thing change the defender’s observation stream? If yes it belongs on the real list and is not there yet.

In my project: Section 4.3.

C2 *“The use of running VMs, real services, real network isolation, SSH-driven reboot operations, and correlated user activity is substantially more realistic than a purely symbolic state-transition simulator. The authors are also appropriately explicit about the boundary between real and emulated components.”*

Review 724C, strength C2

This is credit given by the reviewer who scored a weak reject, for the same artifact Reviewer A4 reads as evidence of a simplistic environment. I do not want to give it back.

without a simulator stage

“a model-based RL framework that trains a defender online and directly in a network on cloud infrastructure, without a simulator stage”

main.tex, abstract

Definition: Every real transition the world model is fitted on comes from the emulation. There is no second environment implementation between the network and the policy.

Distinguishes it from: Sim-to-real pipelines, which fit a policy in one implementation and deploy it in another. The learned world model is not that second implementation: it is fitted from the network it defends and used only for imagined rollouts.

Operational test: Is there a second environment implementation the policy trains against before deployment? One implementation and one fitted model is the claim; two implementations is not.

In my project: Abstract, and Section 2.2.

Simulation

I write that the framework trains “without a simulator stage”, main.tex abstract.

Reviewer A1: “the authors mention that they dont use simulation. However, one could say that the world model generates simulations.”

A1 is not asking for a citation. A1 is saying that CSLE also pairs a data-collection environment with a fitted system model used for optimization and simulation, and is therefore the closest methodological relative. My related work compares CSLE on the substrate axis alone.

A1 *“the CSLE framework uses a dockerized environment for data collection and a system (world) model for optimization and simulation, which is very similar”*

Review 724A, comment A1

the testbed

“The testbed is an eight-host enterprise emulation deployed on an OpenStack cluster.”

main.tex, Testbed

Definition: Eight hosts across three subnets on OpenStack, four classes of CVE-bound vulnerability, GHOSTS user traffic, and a separate controller subnet holding the defender process and the gRPC command-and-control server.

Distinguishes it from: The operational network, which is the stipulated term; the testbed is the thing the term picks out.

Operational test: Can a second copy run beside the first without renaming security objects? The replicas are namespaced per instance for exactly that, and eight run in parallel during collection.

In my project: Sections 4.2 and 4.3.

Three contributions or two

Introduction: “We make three contributions”, the third being the testbed.

Conclusion: “two technical contributions: a factored world model ... and an opponent model”. My own reading notes hold the mentor’s position, two clean technical contributions with the testbed as supporting infrastructure. The introduction and the conclusion currently disagree with each other, and Reviewer A4 and Reviewer C2 read the same artifact in opposite directions, so which sentence survives is a decision and not a copy edit.

the realism ladder

“The tradeoff between simulator fidelity and per-interaction cost can be read as a ladder. Abstract simulators are inexpensive per interaction but unrealistic. Production deployment is realistic but each exploratory action carries operational cost. Higher-fidelity emulation sits in the middle.”

main.tex, Introduction

Definition: An ordering of training environments on two axes at once, fidelity and per-interaction cost, along which the two axes rise together.

Distinguishes it from: The fidelity gap, which is the distance between two points on this ordering. The ladder is the ordering itself.

Operational test: For two environments, which is cheaper per interaction and which is closer to production? If the same environment wins both, they are not on this ladder and one of them is simply better.

In my project: Introduction, and Section 4.3 places the testbed on it.

the fidelity gap

“The fidelity gap between simulator and operational network is the recurring obstacle to deployable autonomous defense.”

main.tex, Introduction

Definition: The distance between where a policy was trained and where it has to work, measured by the properties present in one and absent in the other.

Distinguishes it from: The realism ladder, which is the ordering this gap is measured along. And from the emulation-to-deployment gap, which is the second gap the paper keeps open above its own testbed.

Operational test: Name one property present at deployment and absent in training that changes the defender’s best action. If none can be named, there is no gap to argue about.

In my project: Introduction and Section 2.2, where the CybORG transfer figure of 66 percent is the evidence; Section 7 keeps the second gap open.

depth-over-breadth

“The realism is depth-over-breadth: fewer hosts, more end-to-end fidelity per host.”

main.tex, Testbed, final paragraph

Definition: A stated choice about where the environment budget went. Eight hosts, each carrying real services, real exploit code paths and real consequences, rather than a larger topology of thinner hosts.

Distinguishes it from: Scale-first testbeds. MHBench runs 20 to 50 hosts; this runs 8, and the comparison is a choice rather than a shortfall.

Operational test: Would adding hosts change what the environment can demonstrate? If the claim is about per-host fidelity, no.

In my project: Section 4.3. Reviewer A4 reads the host count as evidence that “the environment is extremely simplistic”; Reviewer C2 credits the same artifact for realism.

behaviorally correlated user activity

“The user behavior is scripted but human-correlated: bursts and pauses follow distributions calibrated to enterprise traces rather than Poisson noise.”

main.tex, Testbed

Definition: Background traffic whose timing structure is drawn from calibrated distributions, so that the intervals between user actions carry the shape real sessions have.

Distinguishes it from: Poisson background noise, which is memoryless. The two evaluation profiles make the difference concrete: U-rand is memoryless and U-burst is not.

Operational test: Are the inter-arrival times drawn independently per step? If yes, the traffic is noise and the entanglement the factoring is built for does not arise.

In my project: Section 4.3, and Section 6.4 uses the two profiles as the ground truth the exogenous latent is decoded against.

per-interaction cost

“Each interaction touches running VMs. Isolation drops real user traffic. Reboots clear attacker persistence and disrupt active sessions.”

main.tex, Motivating Scenario, challenge C3

“Sample efficiency, not asymptotic performance, is the central constraint.”

main.tex, Motivating Scenario, challenge C3

Definition: What one exploratory action costs when the environment is a running network rather than a simulator.

Distinguishes it from: Compute cost. More compute gives more gradient steps on data already collected. This is paid per real transition, and a bigger machine does not recover it.

Operational test: Would one exploratory action have a consequence a user would notice? If yes, the budget is a sample budget.

In my project: Challenge C3, and it is why the argument is about the three day budget rather than about asymptotic performance.

1.4 The evaluation

episode loss

“The primary metric is episode loss, the negative reward sum over the 100-step episode. Lower (more negative) is worse.”

main.tex, Setup

Definition: As above. One number per episode, on the same scale for every method.

Distinguishes it from: The operational floor, which is a value on this scale rather than a different metric. And from a return: this is a loss, so the sign convention inverts the usual reading and every table has to say so.

Operational test: Is the number the negative reward sum over the full 100 steps? A per-step average or a truncated episode is a different quantity.

In my project: Section 6.1, and Tables II and III report it.

matched compute

“Both train on the same observation stream, the same action space, the same compute budget, and the same seeds as FOE-Dreamer. No baseline receives privileged information that FOE-Dreamer does not also see.”

main.tex, Setup

Definition: Four things held equal across methods, named one by one, so the comparison can be checked rather than asserted.

Distinguishes it from: Privileged-information training, which is the one thing not held equal and the reason the sentence exists. The reviewers’ own words for what they wanted checked were baseline fairness and equivalent supervision.

Operational test: Same observation stream, same action space, same compute budget, same seeds. Any single no makes the comparison something other than matched compute, and the sentence has to say which.

In my project: Section 6.1, and Reviewer B5 reads it against Section 5.3.

the operational floor

“The heuristic encodes the standard cyber-deception playbook as a fixed policy and serves as the operational floor that a learned defender is expected to exceed.”

main.tex, Setup

Definition: The score of a rule-based defender that a practitioner could deploy today, used as the value a learned policy has to beat before it is worth anything.

Distinguishes it from: Episode loss, which is the scale this sits on. Sleep and Random sit below it and bracket the range; they are not floors, because nobody would deploy them.

Operational test: Would a practitioner deploy this policy today without training anything? Only then is it a floor.

In my project: Section 6.1. The Expert row is -138 against Avoidant and -88 against Enticed, and FOE-Dreamer passes both.

a testable check on the factoring

“The two tables together give a testable check on the factoring: u_t tracks users and ignores the defender, z_t tracks the defender and ignores the user.”

main.tex, Latent Disentanglement Validation

Definition: A pair of decoding measurements whose joint pattern would come out differently if the factoring were not doing what the architecture says.

Distinguishes it from: Two neighbours. The factor boundary, which is the claim this measurement checks. And the ablation: in my own sentence, the ablations “show that the components help. They do not show whether the components do what we claim or whether they help by some other mechanism.”

Operational test: Would the measurement come out the same if the mechanism were wrong and the performance unchanged? If yes, it is a performance number wearing a diagnostic’s name.

In my project: Section 6.4, Tables IV and V.

E , episodes per iteration

“Run the current policy in E parallel testbed instances for one episode each.”

main.tex, Live Training Loop

Definition: The number of real episodes collected per training iteration. Eight, run as eight parallel instances for one episode each.

Distinguishes it from: The total real interaction over the run, which is E times the number of iterations and is never stated.

Operational test: Is the number per iteration or over the run? If the text does not say, a reader will assume the second.

In my project: Appendix B hyperparameter table, and step 1 of the training loop.

What E counts

The appendix: “Episodes per iteration (E) ...8”. The loop: “Run the current policy in E parallel testbed instances for one episode each”.

Reviewer A4 read it as the total: “they use only 8 episodes for fitting the world model, which seems extremely low”.

Reviewer B6 names the gap that made the misreading available: the paper “reports performance only against wall-clock time, without giving the number of real environment interactions, episodes, or model updates”. The total is never stated anywhere, so A4’s reading is the one the paper supports.

1.5 Where the model is wrong

This vocabulary is on the website and the Next page, not in the paper. It is the language of the follow-on work, and it is the tier I most need fluently, because it is what a reviewer asks about when the results are accepted.

objective mismatch

“objective mismatch, where the model minimises prediction error across the data distribution while the policy needs accuracy only where it will act”

website, FOE-Dreamer page

Definition: As above. The loss is spread over the data distribution while the value of accuracy is concentrated where the policy acts.

Distinguishes it from: Model exploitation. In objective mismatch the error is misallocated inside the training distribution; in model exploitation the policy leaves that distribution.

Operational test: Is the error where the policy will act, or where the policy has gone? The first is mismatch, the second is exploitation.

In my project: Website, FOE-Dreamer page, ongoing work.

model exploitation

“model exploitation, where a policy drifts into states the model never saw and profits from the error there”

website, FOE-Dreamer page

Definition: As above.

Distinguishes it from: Objective mismatch, by which side of the training distribution the error sits on. And, in the adversarial setting, from both of their usual forms: in the model-based RL literature “the drift is an artefact of the training loop. Here it is an opponent’s objective, so uniform predictive loss is selected against.”

Operational test: Is there a process searching for the error, or does the policy find it by accident? An adversary makes it the first.

In my project: Website, FOE-Dreamer page, ongoing work.

confidently wrong

“the adversary is the process that looks for the region where the model is confidently wrong and operates there”

website, Cyber World Modeling, Next

Definition: The region where the model’s error is large and its uncertainty is small. In an adversarial setting this is not a rounding error, because something is looking for it.

Distinguishes it from: Fails honestly against fails silently, which is the preference over error shape that follows from this being the failure that matters.

Operational test: Does the model’s uncertainty rise where its error rises? If the two come apart, the region exists and can be found.

In my project: Next page. The planned outputs are a failure taxonomy for factored world models and a drift detector.

fails honestly against fails silently

“preferring a model that fails honestly (wide and uncertain) over one that fails silently (sharp and wrong)”

website, Cyber World Modeling, Next

Definition: The stated preference over how a model breaks: a wide, uncertain prediction beats a sharp, wrong one at the same error.

Distinguishes it from: Confidently wrong, which names the failure this preference is stated against. And from predictive accuracy, which cannot see the difference: the two failures can have identical loss.

Operational test: At the point of largest error, is the predictive distribution wide or sharp? That question, not the error itself, is the one this preference answers.

In my project: Next page, and it is the criterion the drift detector would be built to.

the fallback

“The fallback is drift detection on the joint latent, which surrenders attribution and keeps the alarm.”

website, FOE-Dreamer page

Definition: What remains if the adversary latent is not identifiable: detection on the joint latent, giving up the claim about which process moved and keeping the claim that something did.

Distinguishes it from: Identifiability of the adversary latent, which is the assumption; this is what is kept when the assumption fails.

Operational test: Can the alarm fire without naming which process caused the drift? If yes, the fallback is available and the factoring is a bonus rather than a precondition.

In my project: Website, FOE-Dreamer page. It is the second half of the answer Reviewer A3 asks for, and it is the half that costs nothing to state.

2 Working

Recognized precisely, lookup acceptable. Definition and discrimination only.

post-initial-access intruder

Definition: An attacker assumed to have already breached the perimeter and to be operating from a foothold on a publicly reachable host. “The defender’s job is to contain expansion before exfiltration, not to prevent the initial breach.”

Distinguishes it from: A perimeter threat model, whose whole budget goes to preventing the first compromise. Everything the defender can do here happens after it.

the five-element defender action space

Definition: Isolate, Disrupt, Monitor, Deceive, Null. Isolate changes OpenStack security-group membership, Disrupt reboots over SSH, Monitor raises log verbosity, Deceive has two variants, and Null lets the step pass.

Distinguishes it from: The attacker’s six-element space, which is drawn from the post-delivery phases of the kill chain and is not the defender’s.

Five actions or six

The paper: “The defender’s action space is five-element”, main.tex Defender Capability.

The paper: “Random-baseline accuracy is 0.17 over six action categories”, main.tex Latent Disentanglement Validation. Five categories give 0.20; six give 0.167.

Deceive “has two variants”, the Cowrie honeypot and the fake SUID binary, which would make six concrete actions. The paper never says the count is taken that way. No reviewer caught this, and it is a one-line fix in the diagnostic caption.

Eradicate

Definition: The episode reset. “episode resets via Eradicate require restoring all eight VMs from snapshot”, which takes 35 to 90 seconds.

Distinguishes it from: Disrupt, which reboots one host inside an episode and is a defender action. Eradicate ends the episode and is an operation of the environment.

Eradicate is used but never defined

Deployment Notes: “episode resets via Eradicate require restoring all eight VMs from snapshot”. Eradicate is not one of the five defender actions and appears nowhere else in the paper. A reader who has just been given a five-element action space meets a sixth capitalized verb with

no definition attached.

three deterministic binary probes

Definition: A banner-fingerprint probe, a credential-validation probe, and a post-exploit verification probe. Each returns SUSPICIOUS or NOT_SUSPICIOUS using attacker-observable information only.

Distinguishes it from: The defender’s Monitor action. The probes are the attacker’s instrument for detecting deception; Monitor is the defender’s instrument for detecting the attacker.

U-rand and U-burst

Definition: The two GHOSTS user profiles used in evaluation. U-rand is memoryless, one independent action per step per user. U-burst follows ON and OFF dynamics with heavy-tail ON durations.

Distinguishes it from: Each other, and the difference is the point: U-burst produces extended periods of concentrated activity that overlap attacker actions in the alert stream, which is the regime where the entanglement is real.

imagined rollouts

Definition: Policy updates taken on trajectories unrolled forward in the learned latent space for $H = 30$ steps, with no real environment steps taken.

Distinguishes it from: The Collect phase, which is the only place real interaction happens. The whole sample-efficiency argument is that most updates live here and not there.

the sliding window, K

Definition: The length of the recent observation history the opponent encoder reads, set to 10.

Distinguishes it from: The recurrent state of the world model, which carries history without a fixed horizon. The window is a fixed horizon on purpose, because the signatures cluster over a short one.

the per-factor KL

Definition: The two Kullback-Leibler terms in the world-model loss, one per factor, with coefficients β_z and β_u tuned independently at 1.0 and 0.5.

Distinguishes it from: A single KL over a joint latent. Two coefficients are what let the exogenous branch be regularized differently from the controllable one, and the setting is not symmetric.

Rainbow and IQN

Definition: The two model-free distributional reinforcement learning baselines, trained on the same stream, action space, compute budget and seeds.

Distinguishes it from: Sleep, Random and the Expert-Inspired Heuristic, which learn nothing and are there to bracket the achievable range rather than to compete.

GHOSTS

Definition: The framework generating the background user population: personas that browse, administer over SSH, query databases and transfer files, with staggered starts and idle gaps.

Distinguishes it from: Kyoushi, which is the closest user emulation in spirit but is a testbed builder rather than a component of a defender gym.

Daedalus

Definition: The name the website gives the environment the paper describes in Section 4.3: an eight-host OpenStack network across three subnets, real services, CVE-bound exploits, GHOSTS traffic.

Distinguishes it from: Nothing in the paper, which never uses the name. The website and the manuscript describe one artifact under two names, and anything written for both audiences has to pick one.

adaptive learning adversaries

Definition: Attackers that retrain against the deployed defender, including attackers that adapt probe timing specifically to fool the opponent encoder. Named out of scope in Section 7.

Distinguishes it from: Fixed-class profiles, which are scripted and do not change. This is the phrase the limitations section already uses, so it is the phrase to use everywhere else.

zero-shot transfer to unseen attacker classes

Definition: Whether the opponent encoder produces useful structure for an attacker class it was never supervised on.

Distinguishes it from: Two-way separation, which is what the current evidence shows. Reviewer B6 asks “whether the opponent model learns transferable behavior or simply distinguishes two known scripts”, and Reviewer C6 writes that “the current results mainly show that the encoder can distinguish the two supervised attacker classes”. Two reviewers, one point, stated twice: both accept that it separates and doubt that separating is the same as modeling.

3 Peripheral

Know approximately where each belongs and retrieve it when needed. Definition and a pointer.

CVSS

Definition: The severity score attached to a vulnerability. The WordPress Backup Migration flaw at 9.8 is the canonical entry vector here.

Where it belongs: Section 4.3, Table I. Three of the four deployed vulnerability classes carry no CVSS at all, which is why the table has an N/A column.

Cowrie and the fake SUID binary

Definition: The two variants of the Deceive action: an SSH honeypot bound to weak credentials, and a planted binary intended to attract privilege-escalation attempts.

Where it belongs: Section 4.2, and the five-or-six action count above.

OpenStack security group

Definition: The cloud-level access rule set whose membership Isolate changes, which is how quarantine is implemented.

Where it belongs: Section 4.2, and the deployment notes, where Isolate completes in under a second while SSH-driven Disrupt takes 2 to 6.

the kill chain, post-delivery phases

Definition: The attacker's six-element action space: Discover, Compromise, Escalate, Exfiltrate, Detect Deception, Null.

Where it belongs: Section 4.1, and the motivating scenario walks one path through it.

the C2 server

Definition: The gRPC service in the controller subnet that translates defensive tactics into OpenStack API calls and SSH operations downward, and sensor data into structured observations upward.

Where it belongs: Section 4.3, Figure 3. One endpoint per parallel replica.

CybORG and the CAGE Challenge

Definition: The dominant simulator lineage in autonomous cyber defense, modeling discrete kill-chain steps.

Where it belongs: Section 2.2, and the 66 percent simulation-to-emulation transfer figure comes from the original CybORG report.

CSLE

Definition: A Docker-based emulation with CVE-tagged containers and a reinforcement learning polling loop, without OpenStack and without GHOSTS-class user behaviour.

Where it belongs: Section 2.6, and it is the comparison Reviewer A1 raises, so the entry that matters is under *without a simulator stage* above.

MHBench and Incalmo

Definition: OpenStack-backed multi-host benchmarks for language-model red teams, with real CVEs, no user emulation, and no reinforcement learning interface.

Where it belongs: Section 2.6, and Section 4.3 uses MHBench for the scale comparison behind depth-over-breadth.

Cyberwheel, Yawning Titan, CyberBattleSim, NetSecGame

Definition: The rest of the environment field. Cyberwheel is an emulator on FIREWHEEL and QEMU; the other three are simulators abstracting a different slice each.

Where it belongs: Sections 2.2 and 2.6, where the three groups of prior autonomous-defense work are set out.

Kyoushi

Definition: State-machine user emulation, closest in spirit to GHOSTS, but a testbed builder rather than a gym.

Where it belongs: Section 2.6, on the user-emulation axis of the four-axis comparison.

t-SNE and ridge regression

Definition: The two instruments behind the disentanglement figures: a projection for looking at the opponent embedding, and a linear fit for decoding user activity out of each latent.

Where it belongs: Section 6.4. Both are deliberately weak instruments, which is what makes the numbers a check rather than a demonstration.

DreamerV3

Definition: The world-model family this architecture is built inside, and the source of every architectural choice not stated in Section 5.

Where it belongs: Section 2.4 and Appendix B.

Denoised MDP and IFactor

Definition: The two prior factorings of latent dynamics that split on controllability and reward relevance.

Where it belongs: The foundations work rather than the paper. IFactor feeds the uncontrollable factor to the policy and this architecture does not, which is a real disagreement and the place to start when the factoring has to be defended.

4 The two checks

4.1 Discrimination, run in both directions

If one entry names a second as its neighbour, the second names the first. A one-way distinction usually means one of the two definitions is wrong. Thirteen pairs come back both ways.

Entry	Its neighbour
controllable latent z_t	exogenous latent u_t
exogenous latent u_t	opponent embedding τ
the factor boundary	the architectural commitment
the factor boundary	a testable check on the factoring
signatures	probe-and-withdraw
privileged-information training	matched compute
fixed-class profiles	adaptive learning adversaries
operational network	the testbed
the realism ladder	the fidelity gap
episode loss	the operational floor
objective mismatch	model exploitation
confidently wrong	fails honestly against fails silently
identifiability of the adversary latent	the fallback

One distinction runs one way only, and it survived the check rather than failing it. Identifiability names a testable check on the factoring as its second neighbour; the check does not name identifiability back, because from inside the measurement the assumption is invisible. That is the asymmetry between an assumption and an instrument, and it is worth leaving in.

Seven Active entries name a neighbour that is no headword here, which is correct in every case because the neighbour is a term from the field rather than from this project: factored world model against a monolithic-state world model, spurious credit assignment against reward noise, without a simulator stage against sim-to-real transfer, the boundary between what is real and what is emulated against a fidelity claim, depth-over-breadth against scale-first testbeds, behaviorally correlated user activity against Poisson noise, and per-interaction cost against compute cost.

4.2 Where two locators disagree

Eight entries cite two files that use one term differently. Each sits in the entry it belongs to rather than in a list of errata, because it is a part of the concept that conflicts with another part, and writing it down is what keeps it from being met for the first time in a review.

The disagreement	The entry it sits in
τ_t or z_t^{opp} , observations or latents	opponent embedding τ
Two decoders or one classifier head	privileged-information training
Whether τ reaches the observation decoder	factored world model
Five actions or six	the five-element defender action space
Eradicate, used and never defined	Eradicate
Three contributions or two	the testbed
What E counts	E , episodes per iteration
Simulation, and what the world model generates	without a simulator stage

Three of the eight are the same problem seen from three sides: the opponent representation’s input,

its training target, and where it is read. Reviewer B7 asks about all three in one sentence, so they are answered together or not at all.

5 What crosses into the shared lexicon

Terms this project uses that the whole Cyber World Modeling cluster uses. Their cluster sense goes in the shared file and the entries above carry only what FOE-Dreamer fixes about them: world model, factoring, latent, opponent latent, opponent model, drift, non-stationary, identifiability, objective mismatch, model exploitation, fails honestly against fails silently, and zero-shot and few-shot transfer.

Two of those already differ between projects and stay in both files. Drift on this page means an opponent latent moving away from what the model was fitted on; in the model-based reinforcement learning sense quoted above it means a policy leaving the data distribution. Opponent has a second sense in Learn Structure, where the question is how much a best action turns on the other player rather than which class the other player belongs to. That difference is a finding, not a defect to be normalized away.

6 Sources

- Every definition and every quoted sentence in the Active tier is from `[May26th]_Full/main.tex` and its sections, or from the FOE-Dreamer, Cyber World Modeling and Next pages of the website. The locator on each entry says which.
- Reviewer sentences are from the three reviews for submission 724, quoted as sent.
- The three tiers, the three fluency levels and the four fields are from the Tech Fluency page. The reason the second field exists, and the reason a term whose two uses disagree belongs inside its entry, are in `_side/_lexicons/lexicon_method.tex`.
- The census this was built from is `_side/_lexicons/your_terms_foedreamer.tex`.